

Lecture : DP in practice - I: Frequency estimation under local DP

In this sequence of lectures, we shall take a look at some examples of DP deployments in practice — particularly, by tech giants such as Google & Apple and by the US Census Bureau.

The first such example will be that of frequency estimation / estimation of heavy-hitters [items that occur very frequently] in data mined from users. The "items" here could refer to events such as malware hijacks in browsers [RAPPOR (2014)] or word abbreviations ("wyd", "wbu", "bruh") and emojis typed by users [Learning with privacy at scale (2017), Apple], or websites that demand high energy/memory.

More concretely, consider the setting where there are N users, each with an item $v_i \in [d]$; we are interested in estimating the vector $f = (f(x) : x \in [d])$, privately, where

$$f(x) = \sum_{i=1}^N \mathbb{1}\{v_i = x\}.$$

In particular, our estimator $\hat{f}$ of f must be such that

$$\max_{x \in [d]} |\hat{f}(x) - f(x)| \text{ is small.}$$

Our first mechanism shall construct an ϵ -DP estimator $\hat{f}$, via estimator of "one-hot encodings" of $(v_i : i \in [N])$.

Algorithm 1 (User)

→ For each $i \in [N]$, construct the one-hot encoding of v_i , i.e., the vector $V_i \in \{0,1\}^d$ such that $V_{i,v_i} = 1$ and $V_{i,j} = 0$, for $j \neq v_i$.

→ Given the vector V_i , perform a variant of randomized response:

For each coordinate $j \in [d]$:

(a) If $V_{i,j} = 0$, set $\bar{V}_{i,j} \sim \text{Unif}(\{+1, -1\})$.

(b) If $V_{i,j} = 1$, set $\bar{V}_{i,j} = \begin{cases} +1, & \text{w.p. } \frac{1+\epsilon}{2}, \\ -1, & \text{w.p. } \frac{1-\epsilon}{2}. \end{cases}$

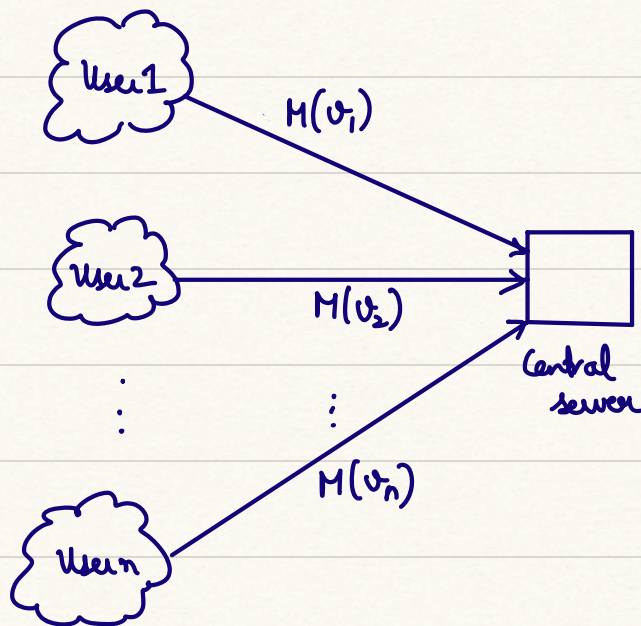
→ Transmit $\bar{V}_i$ to the server.

Note that we wish to prove that Algorithm 1 is ϵ -DP in the local model (i.e., that Algorithm 1 is ϵ -LDP). We shall first put down a formal definition of ϵ -LDP.

Def 1 (ϵ -LDP): An ϵ -LDP algorithm $M: V \rightarrow \mathcal{Y}$ is s.t.
 $\forall v, v' \in V$, and for all $y \in \mathcal{Y}$, we have

$$P[M(v)=y] \leq e^\epsilon \cdot P[M(v')=y].$$

Remark: The chief difference b/w ϵ -LDP and ϵ -DP is that we would like the indistinguishability above to hold for only "neighbouring" datasets in standard (central) DP, but we now require, under LDP, that this indistinguishability hold for all inputs [think of the algorithm as now being executed by each user (see below) instead of by a central server].



The LDP framework is useful in those settings where the central server is not trustworthy.

HW: For what value of ϵ is Algorithm 1 ϵ -LDP?

Now that the users have "privatized" their inputs, we need to argue that the server can extract a high fidelity estimate of f .

Algorithm 2 (Server)

→ Given $\bar{V}_1, \dots, \bar{V}_n$, the server computes

$$\hat{f} = \frac{1}{\epsilon} \sum_{i=1}^n \bar{V}_i.$$

Claim: We have that

$$\Pr \left[\max_{x \in [d]} |f(x) - \hat{f}(x)| \geq \Omega\left(\sqrt{n \log(d/\beta)}\right) \right] \leq 1 - \beta,$$

for any $\beta \in (0, 1)$.

Proof: Observe that $\forall x \in [d]$,

$$\hat{f}(x) = \frac{1}{\epsilon} \left(\sum_{i: v_i = x} Z_i + \sum_{i: v_i \neq x} Z_i' \right),$$

where $Z_i \sim \text{Ber}_{\pm 1}\left(\frac{1+\bar{\epsilon}}{2}\right)$ and $Z_i' \sim \text{Unif}(\pm 1)$. Via a direct calculation, we see that $\mathbb{E}[Z_i'] = 0$ and $\mathbb{E}[Z_i] = \bar{\epsilon}$, showing that $\mathbb{E}[\hat{f}(x)] = \#\{i: v_i = x\} = f(x)$.

Furthermore, via a variant of Hoeffding's inequality, we have that for RVs $X_1, \dots, X_n$ s.t. $X_i \in [-1, 1] \forall i \in [n]$,

$$\Pr \left[\left| \frac{1}{n} \sum_{i=1}^n (X_i - \mathbb{E} X_i) \right| \geq t \right] \leq 2 \exp\left(-\frac{nt^2}{2}\right).$$

Thus, since $z_i, z_j' \in [-1, 1]$, for all i, j , we have that for any $x \in [d]$,

$$\Pr \left[|f(x) - \hat{f}(x)| \geq \sqrt{2n \log\left(\frac{2d}{\beta}\right)} \right] \leq 2 \cdot \exp\left(-\frac{n}{2} \frac{2}{n} \log\left(\frac{2d}{\beta}\right)\right) = \beta_d.$$

Thus, via a union bound, we have

$$\Pr \left[|f(x) - \hat{f}(x)| \geq \sqrt{2n \log\left(\frac{2d}{\beta}\right)}, \text{ for some } x \in [d] \right] \leq \beta,$$

$$\text{implying that } \Pr \left[\max_x |f(x) - \hat{f}(x)| \leq \sqrt{2n \log\left(\frac{2d}{\beta}\right)} \right] \geq 1 - \beta \quad \square$$

However, this implementation of LDP frequency estimation suffers from the drawback of high memory and communication costs for the users and for the server. Certain deployment ([\[Apple \(2017\)\]](#)) seek to overcome this using more sophisticated methods relying on the count sketch data structure.

Preliminaries: The count sketch data structure

The count sketch data structure attempts to provide "compressed" storage of the frequencies of items. A key tool this data structure employs is **hash functions**; in particular, pairwise independent hash functions.

Def 2: A family of functions $\mathcal{H} = \{h: h: [n] \rightarrow [m]\}$ is a family of pairwise indep. hash functions if $\forall i \neq j \in [n]$ and $k, l \in [m]$,

$$\Pr_{h \sim \text{Unif}(\mathcal{H})} [h(i) = k, h(j) = l] = \frac{1}{m^2}.$$

HW: Argue that this means that $\Pr_h [h(i) = h(j)] = \frac{1}{m}$ and that $\Pr [h(i) = k] = \frac{1}{m}$.

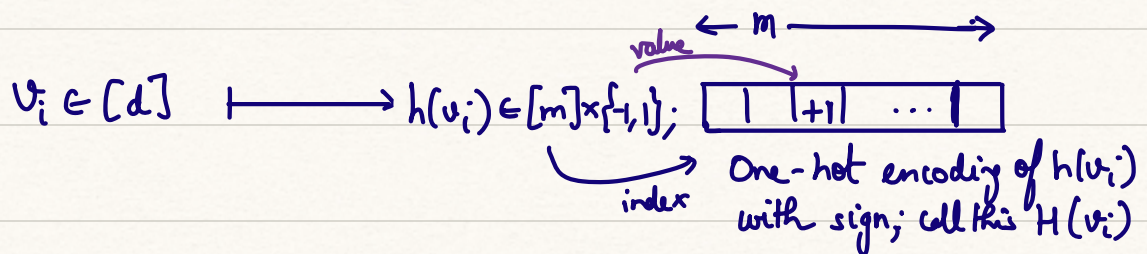
Example: Let p be prime. For $a, b \in \mathbb{Z}_p$, define $h_{a,b}(x) = a \cdot x + b$ (where multiplication, addition are modulo p). Then, $\mathcal{H} = \{h_{a,b} : a, b \in \mathbb{Z}_p\}$ forms a pairwise independent hash family.

To see why, note that

$$\begin{aligned} & \Pr_{a,b \in \mathbb{Z}_p} [h_{a,b}(x) = k \text{ and } h_{a,b}(y) = l] \\ &= \Pr [ax + b = k, ay + b = l] \\ &= \Pr \left[a = \frac{k-l}{x-y}, b = \frac{lx-ky}{x-y} \right] = \frac{1}{m^2}. \end{aligned}$$

$$k - \underbrace{(k-l)}_{x-y} = \underbrace{lx-ky}_{x-y} + l$$

In the count sketch procedure, each user maps



Lemma: We have that if $v = v'$, then $\overbrace{H(v) \cdot H(v')}^{\text{dot product}} = 1$. Else,

$$H(v) \cdot H(v') = \begin{cases} 0, & \text{w.p. } 1 - \frac{1}{m} \\ \pm 1, & \text{w.p. } \frac{1}{2m} \text{ each.} \end{cases}$$

Proof: HW

At the server end, one computes $\underline{u} = \sum_{i=1}^n H(v_i)$ and extracts $\hat{f}(x) \triangleq H(x) \cdot \underline{u}$.

Claim: $\hat{f} = (f(x): x \in [d])$ is an unbiased estimator of f .

Proof: Note that

$$\hat{f}(x) = \underbrace{\sum_{i: v_i = x} H(x) \cdot H(v_i)}_{f(x), \text{ by lemma above}} + \sum_{i: v_i \neq x} H(x) \cdot H(v_i).$$

By another application of the lemma above, we have $\mathbb{E}[\hat{f}(x)] = f(x)$. $\square$

FACT: One can use concentration properties to argue that for $m \geq 2(\sqrt{n})$, we have $\forall x, |\hat{f}(x) - f(x)| \leq O(\sqrt{n})$.

In practice, Apple uses a count-sketch datastructure in combination with a tree-based frequency estimator as a "compressed" estimate of frequencies, which are then privatized via the variant of RR we saw earlier.

[Project reading: Local (distributed) DP]

algorithms, from Reef Bascity's collection].